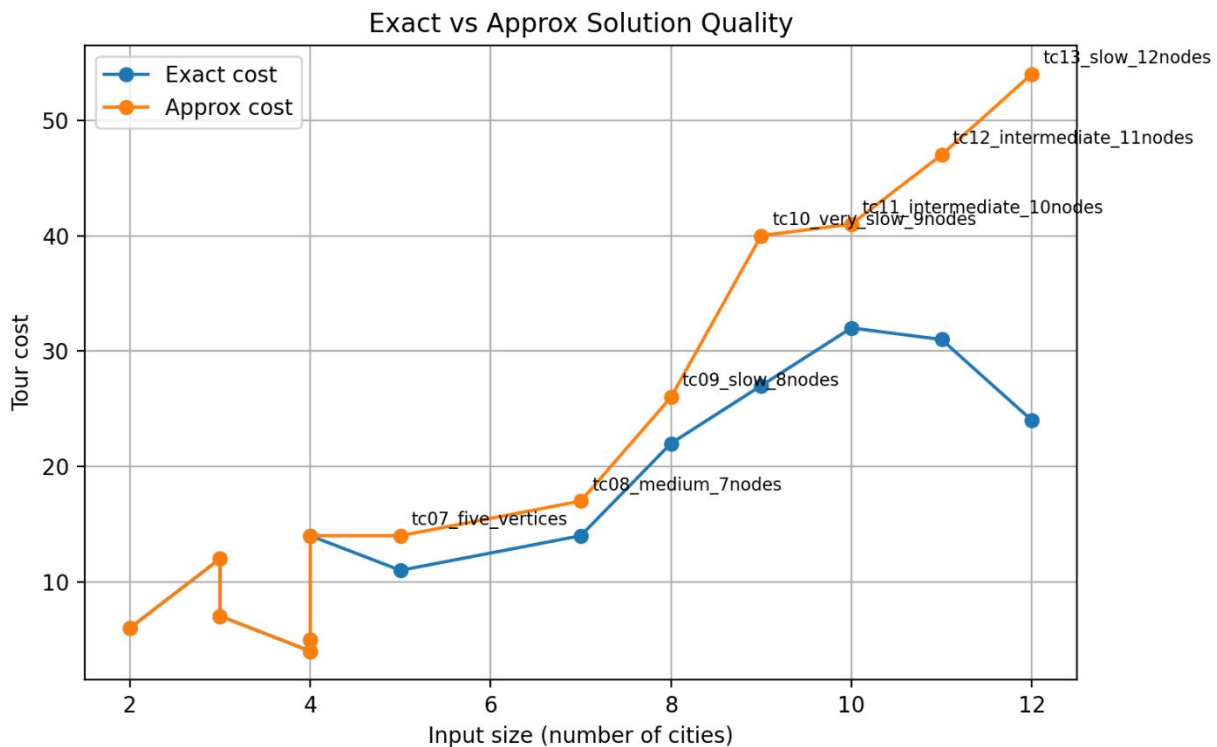


Strategy and Approach:

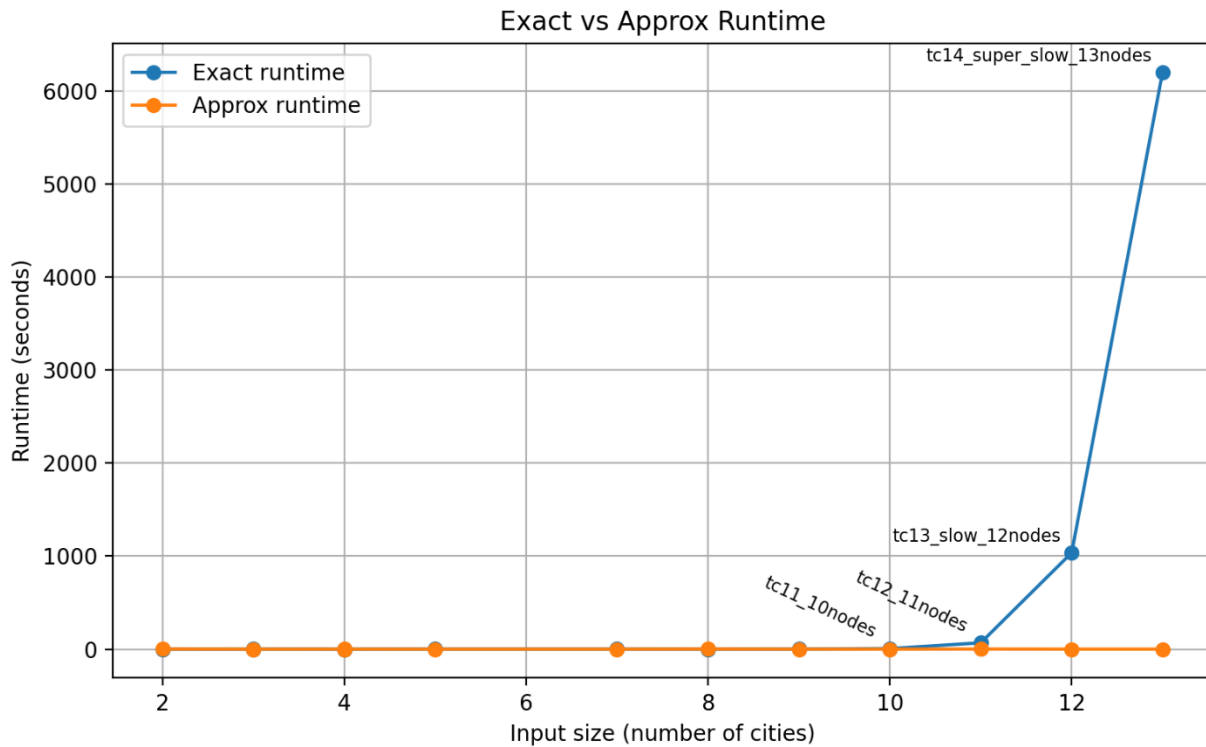
Chooses a random vertex and does a stochastic greedy (nearest neighbor) algorithm several times. When the input size is less than 200, it runs the program 10 more times, when its less than 600 it runs 8 times, and when less than 1000 it runs 4 times, anything greater than 1000 it runs 3 times. This approach allows me to keep my extremely fast runtime while still sacrificing a little bit of time for slightly more accuracy.

In the test cases ran between approx. and exact, I found that approx. begins to fall behind exact in optimizing tour cost around 7 nodes – 10 nodes. There isn't a hard value where it begins to perform worse due to the randomness aspect of my implementation. However, the plot below show approx. vs exact in cost:



The benefits of my approx. algorithm becomes more obvious as the # of inputs increase. Due to exact having a factorial runtime and my stochastic greedy approach being quadratic runtime.

Below is a plot demonstrating that:



Empirically, the runtime grows quadratically with the number of vertices.

Because the test cases contain only up to 13 cities, the algorithm executes extremely quickly, and due to the wall-clock restraints at low times (0 or 1 seconds) it limits precision.

However, as n increases, we see more 1-second runtimes, matching with the $O(n^2)$ behavior of the stochastic greedy algorithm.